

Introduction to Deep Learning with PyTorch

Pytorch로 배우는 딥러닝 기초

PyTorch 기본 문법

PyTorch 소개



$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d}})V \quad (2)$$

where d is the number of columns of Q and K . In these work [35, 12, 7], they all use the multi-head attention, as introduced in [35],

$$MultiHeadAttention(Q', K', V') = Concat(head_1, \dots, head_h)W^O \quad (3)$$

where $head_i = Attention(Q'W_i^Q, K'W_i^K, V'W_i^V)$

딥러닝 기법인 Attention 기법을 설명하는 수식 중 일부

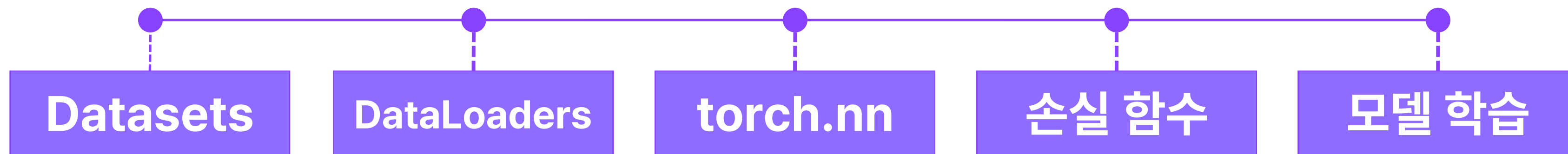
딥러닝 모델을 처음부터 구현하기 위해서는 행렬, 선형 대수학 등 수학에 대한 깊은 이해가 필요하다. 그러나, 딥러닝 프레임워크를 활용하면 수학에 대한 깊은 지식이 없어도 딥러닝 모델을 구현할 수 있다. 딥러닝 프레임워크는 딥러닝 학습 전 과정을 쉽게 진행할 수 있도록 다양한 도구를 제공한다.



- Torch 기반의 오픈소스 머신러닝 라이브러리로 컴퓨터 비전과 자연어 처리 분야에 유용하게 사용
- Meta의 AI 연구소에서 Torch 라이브러리를 파이썬 기반으로 개발하여 PyTorch라 이름 붙임
- 연구 분야에서 거의 80% 이상의 논문이 PyTorch를 사용하였음

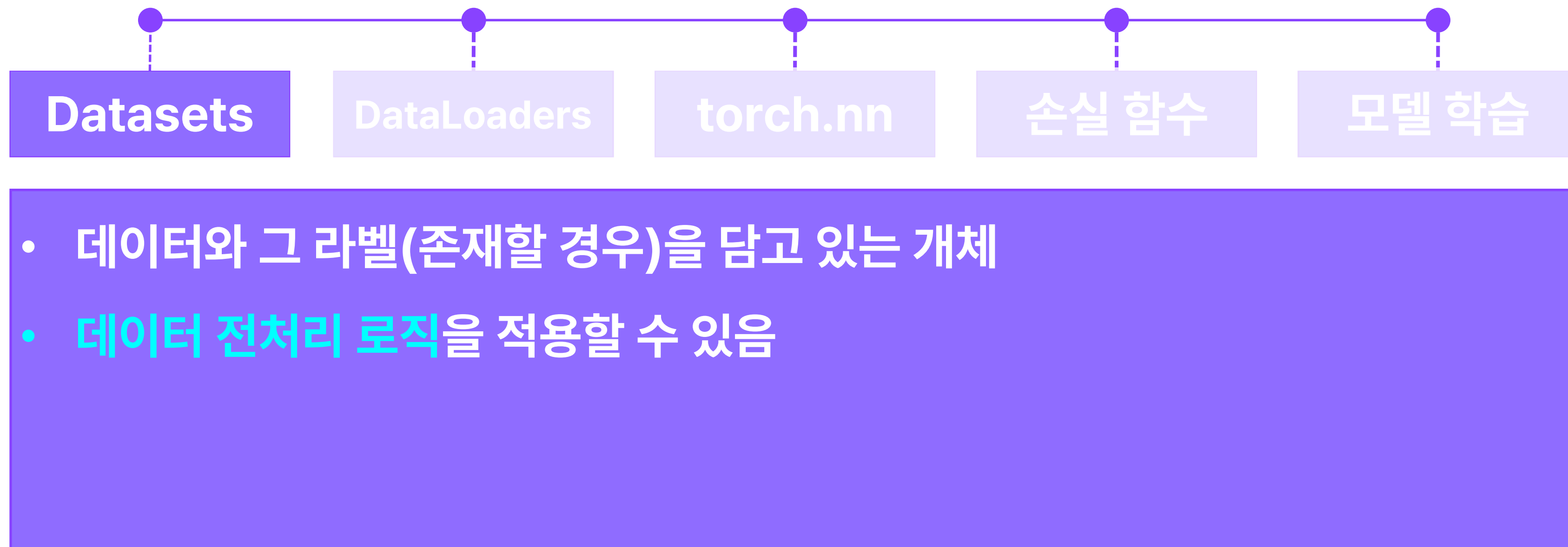


PyTorch 주요 키워드



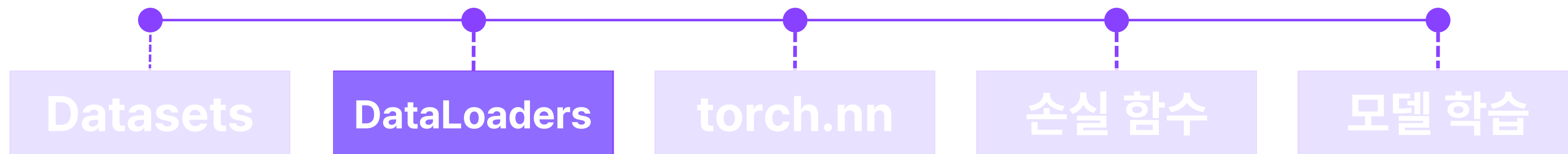


PyTorch 주요 키워드





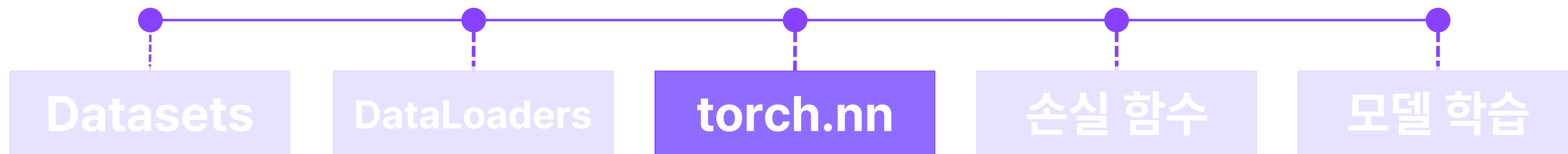
PyTorch 주요 키워드



- 학습 및 테스트 과정에서 **Dataset을 순회하면서** 데이터를 제공하는 개체
- Mini-batch 구성 등 데이터셋에 대한 다양한 로직을 적용할 수 있음.



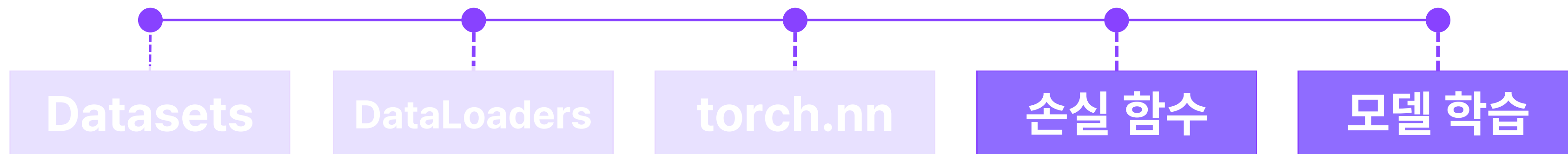
PyTorch 주요 키워드



- 딥러닝 모델을 구성하기 위한 여러 레이어가 구현되어 있는 공간
- torch.nn 에 있는 다양한 레이어를 활용해 딥러닝 모델을 구현할 수 있음



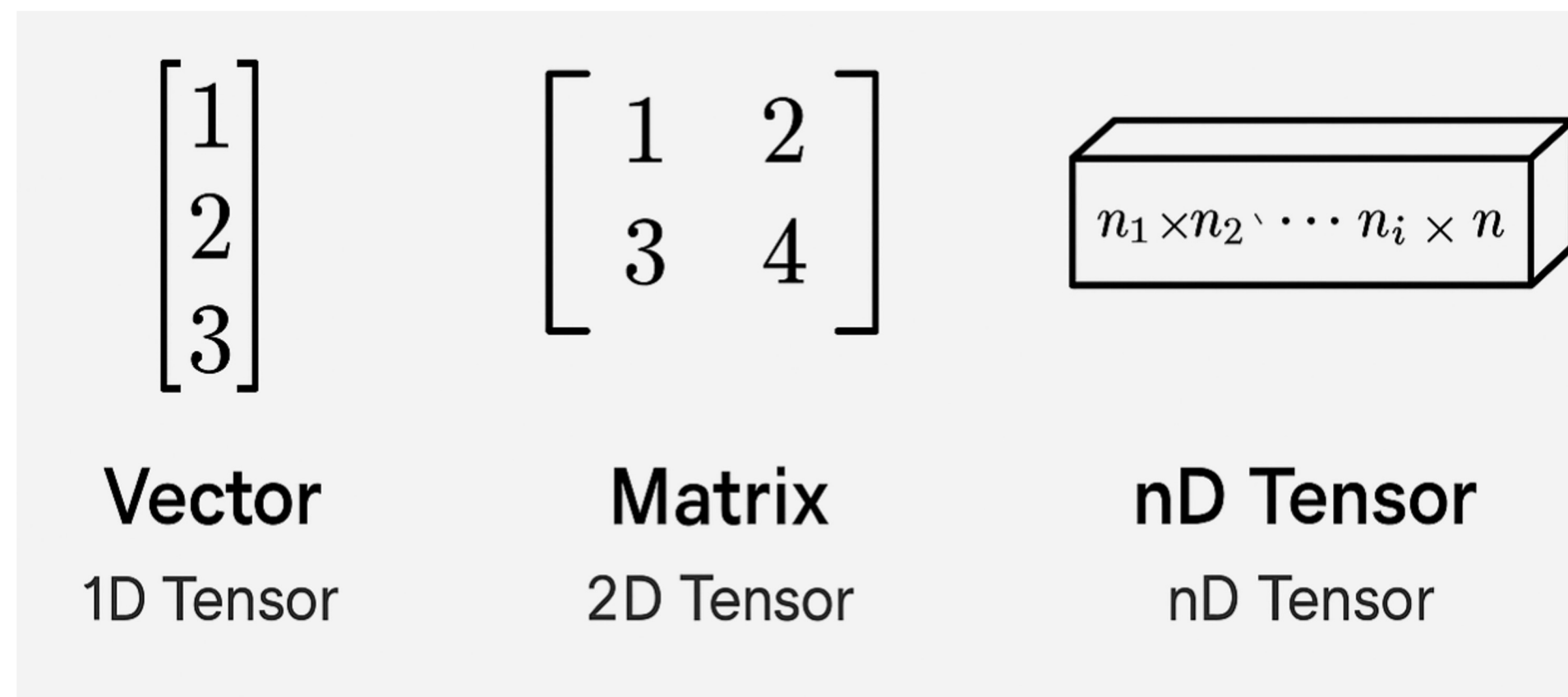
PyTorch 주요 키워드



- Adam, SGD를 비롯한 다양한 손실 함수를 모델 학습에 활용 가능
- DataLoader와 손실 함수를 바탕으로 학습

PyTorch Tensor 기초 문법

- **Tensor**는 PyTorch에서 데이터를 표현하는 기본 단위이며, 인공지능망 학습의 전 과정에서 필수적으로 사용된다.
- 수학적으로 Tensor는 다차원 배열 (n차원 행렬) 로 이해할 수 있다.
- PyTorch에서도 Tensor와 Numpy 배열 간 상호 변환이 가능하다.





Tensor와 Numpy의 관계

```
import numpy as np

arr = np.array([[1, 2, 3]])
t = torch.from_numpy(arr)

print("Tensor:", t)
# Tensor: tensor([[1, 2, 3]], dtype=torch.int32)

print("Back to Numpy:", t.numpy())
# Back to Numpy: [[1 2 3]]
```

torch.from_numpy()와 **numpy()**메서드를 활용하여 Tensor와 Numpy 배열 간 상호 변환이 가능하다.

- 단, 공유된 메모리를 사용하므로 한 쪽에서 변경한 값이 다른 쪽에도 영향을 줄 수 있음에 유의한다.



Tensor 생성

```
import torch

# 리스트로부터 생성
t1 = torch.tensor([1.0, 2.0, 3.0])
print("t1:", t1)
# t1: tensor([1., 2., 3.])

# 0으로 채워진 텐서
t2 = torch.zeros((2, 3))
print("t2:", t2)
# t2: tensor([[0., 0., 0.],
#             [0., 0., 0.]])

# 평균 0, 표준편차 1의 정규분포 텐서
t3 = torch.randn((2, 2))
print("t3:", t3)
# t3: tensor([[ 0.6208,  0.0195],
#             [-0.8228, -0.0672]])
```

PyTorch에서 Tensor를 생성하는 다양한 방법

torch.tensor()를 사용하여 리스트로부터 Tensor를 직접 생성할 수 있다.

torch.zeros()나 **torch.ones()**를 이용하면 각각 0, 1로 채워진 Tensor를 구성할 수 있다.

torch.randn()은 정규분포를 따르는 무작위 값으로 구성된 Tensor를 생성할 수 있다.



Tensor 속성

```
x = torch.rand((3, 4), dtype=torch.float32, device='cpu')

print("Shape:", x.shape)
# Shape: torch.Size([3, 4])

print("Data type:", x.dtype)
# Data type: torch.float32

print("Device:", x.device)
# Device: cpu

x.to("cuda:0") # GPU로 이동
```

Tensor 객체의 3가지 주요 속성

shape는 Tensor의 크기와 차원을 나타낸다.

dtype은 Tensor에 저장되는 데이터의 타입을 의미한다.

device는 이 Tensor가 CPU에 있는지, 혹은 GPU에 있는지를 나타낸다.

- PyTorch는 GPU 가속 연산을 지원하므로 GPU를 사용할 수 있는 경우, device 설정은 성능에 매우 중요한 요소이다.



Tensor 연산

```
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[10, 20], [30, 40]])

# 덧셈, 곱셈, 행렬곱
print("Add:\n", a + b)
# Add:
# tensor([[11, 22],
#         [33, 44]])

print("Element-wise Multiply:\n", a * b)
# Element-wise Multiply:
# tensor([[ 10,  40],
#         [ 90, 160]])

print("Matrix Multiply:\n", a @ b)
# Matrix Multiply:
# tensor([[ 70, 100],
#         [150, 220]])
```

- Tensor는 덧셈, 곱셈, 행렬곱과 같은 수치 연산을 지원한다.
- +, *, @ 연산자나 **torch.add()**, **torch.matmul()** 같은 함수들을 사용할 수 있다.

이 연산들은 역전파 과정을 위한 자동 미분이 가능한 연산 그래프를 형성하게 되며, 이후 학습 시 사용된다.



Tensor 연산

```
# matmul
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[10], [20]])

print("Add:\n", torch.add(a, b))
# Add:
# tensor([[11, 12],
#         [23, 24]])

print("Matmul:\n", torch.matmul(a, b))
# Matmul:
# tensor([[ 50],
#         [110]])
```

- Tensor는 덧셈, 곱셈, 행렬곱과 같은 수치 연산을 지원한다.
- `+`, `*`, `@` 연산자나 **`torch.add()`**, **`torch.matmul()`** 같은 함수들을 사용할 수 있다.

이 연산들은 역전파 과정을 위한 자동 미분이 가능한 연산 그래프를 형성하게 되며, 이후 학습 시 사용된다.



Tensor 차원

```
x = torch.randn(1, 3, 1, 5)
print("Original shape:", x.shape)
# Original shape: torch.Size([1, 3, 1, 5])

# 차원 제거
x_squeezed = x.squeeze()
print("After squeeze:", x_squeezed.shape)
# After squeeze: torch.Size([3, 5])

# 차원 추가
x_unsqueezed = x_squeezed.unsqueeze(0)
print("After unsqueeze:", x_unsqueezed.shape)
# After unsqueeze: torch.Size([1, 3, 5])

# 모양 변경 (reshape)
x_viewed = x.view(-1, 5) # -1은 크기를 자동 계산
print("After view:", x_viewed.shape)
# After view: torch.Size([3, 5])
```

Tensor는 다양한 차원으로 구성되며, 차원을 자유롭게 추가하거나 제거할 수 있다.

tensor.shape: 텐서의 모양 확인

squeeze(): 차원이 1인 부분 제거

unsqueeze(dim): 특정 위치에 차원 추가

view(): 텐서의 크기를 재조정

Tensor 차원 확장

```
a = torch.tensor([[1], [2], [3]]) # 3x1
b = torch.tensor([40, 50, 60])    # 1x3

# 브로드캐스팅되어 3x3 연산 수행
result = a + b
print("Broadcasted result:\n", result)
# Broadcasted result:
# tensor([[41, 51, 61],
#         [42, 52, 62],
#         [43, 53, 63]])
```

- PyTorch는 자동으로 차원을 확장하는 브로드캐스팅을 지원한다.
- 이를 활용하여 서로 크기가 다른 일부 Tensor도 연산이 가능하다.



PyTorch 역전파 연산 (Autograd)



```
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[10, 20], [30, 40]])

# 덧셈, 곱셈, 행렬곱
print("Add:\n", a + b)
# Add:
# tensor([[11, 22],
#         [33, 44]])

print("Element-wise Multiply:\n", a * b)
# Element-wise Multiply:
# tensor([[ 10,  40],
#         [ 90, 160]])

print("Matrix Multiply:\n", a @ b)
# Matrix Multiply:
# tensor([[ 70, 100],
#         [150, 220]])
```

- Tensor는 덧셈, 곱셈, 행렬곱과 같은 수치 연산을 지원한다.
- +, *, @ 연산자나 **torch.add()**, **torch.matmul()** 같은 함수들을 사용할 수 있다.

이 연산들은 역전파 과정을 위한 자동 미분이 가능한 연산 그래프를 형성하게 되며, 이후 학습 시 사용된다.


```
a = torch.tensor([3.0], requires_grad=True)
b = torch.tensor([2.0], requires_grad=True)

z = a * b + b ** 2  # z = ab + b^2
z.backward()

print("dz/da:", a.grad)  # b = 2
print("dz/db:", b.grad)  # a + 2b = 3 + 4 = 7
# dz/da: tensor([2.])
# dz/db: tensor([7.])
```

PyTorch Autograd는 Tensor 연산 과정에서 생성된 연산 그래프를 활용하여 **자동으로 역전파 연산과 가중치 업데이트를 수행하는 기능**이다.

- Tensor는 **requires_grad** 속성을 가질 수 있다.
requires_grad가 True로 설정된 Tensor는 연산 기록을 저장하며, 이후 최적화 함수의 **.backward()** 호출 시 gradient가 자동 계산된다.



torch.no_grad() 블록

```
x = torch.tensor([5.0], requires_grad=True)

with torch.no_grad():
    y = x ** 2

print("Requires grad?", y.requires_grad)  # False
```

- 모델 검증과 같이 역전파 연산이 필요 없는 경우 **with torch.no_grad()** 블록을 통해 연산 기록을 하지 않는 블록을 설정할 수 있다.
- 해당 블록에서 새로 생성된 Tensor는 **requires_grad** 값이 False로 설정된다.



신경망 모델의 requires_grad 속성

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.softmax(self.fc3(x))
        return x

model = MLP()
```

다음 챕터에서 학습할 신경망 모델의 각 **레이어의**
가중치는 대부분 `requires_grad=True`로
설정된 Tensor로 구성된다.



신경망 모델의 검증 모드

```
model = MLP()
```

```
# 학습 코드와 같은 for loop에서 각 epoch 마다 실행되는 검증 코드
model.eval() # 모델을 검증 모드로 설정
val_loss = 0
correct = 0
with torch.no_grad(): # 최적화를 진행하지 않도록 환경 설정
    for images, labels in val_loader: # DataLoader
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        val_loss += criterion(outputs, labels).item()
        pred = outputs.argmax(dim=1, keepdim=True)
        correct += pred.eq(labels.view_as(pred)).sum().item()
```

모델 검증을 위해 **eval()** 메소드를 호출하면, 해당 모델의 가중치의 **requires_grad**를 **False로 설정**하고, Dropout 등 학습 과정에서 사용되는 일부 기능을 비활성화 한다.



신경망 모델의 학습 모드

```
model = MLP()
```

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```

학습 과정에서는 **train()** 메서드를 호출해

가중치를 **requires_grad=True**로 설정하고, Dropout 등 일부 기능을 다시 활성화 한다.